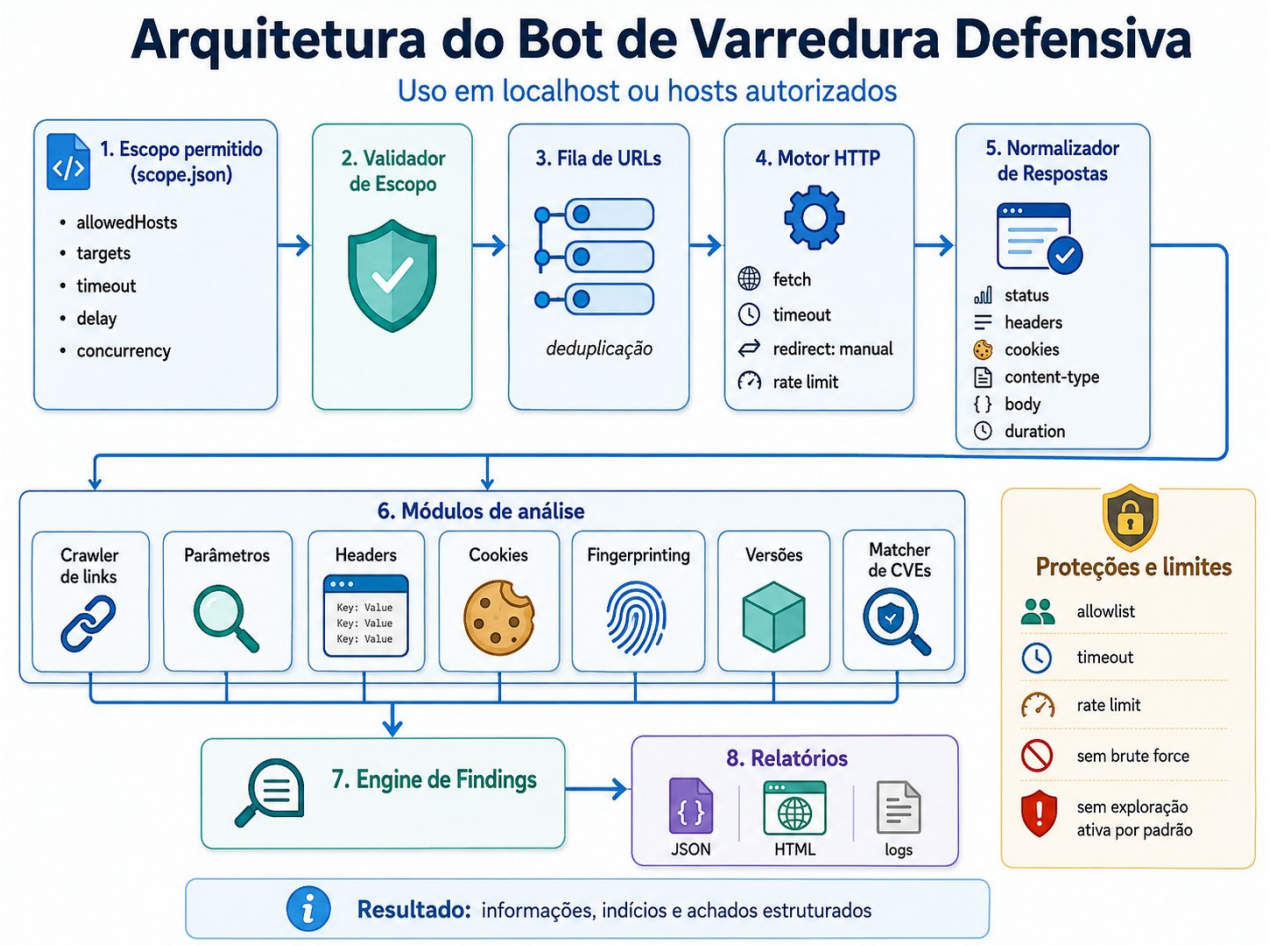
ETHICAL HACKING & APPSEC

Aulas 36 a 40

Threat Modeling • Secure Coding • SAST/DAST/SCA • DevSecOps • Projeto Final

Material de estudo com exemplos em JavaScript/Node.js e tradução conceitual para Portugol. Uso didático e defensivo em laboratório ou ambientes autorizados.



Arquitetura do scanner defensivo usado como projeto paralelo do curso.

Observação: Portugol é usado para explicar a lógica. Bibliotecas como Zod, bcrypt, Pino e drivers de banco não possuem equivalentes nativos em Portugol.

Conteúdo

Aula 36 - Threat Modeling com STRIDE

Aula 37 - Secure Coding

Aula 38 - SAST, DAST, SCA e Secret Scanning

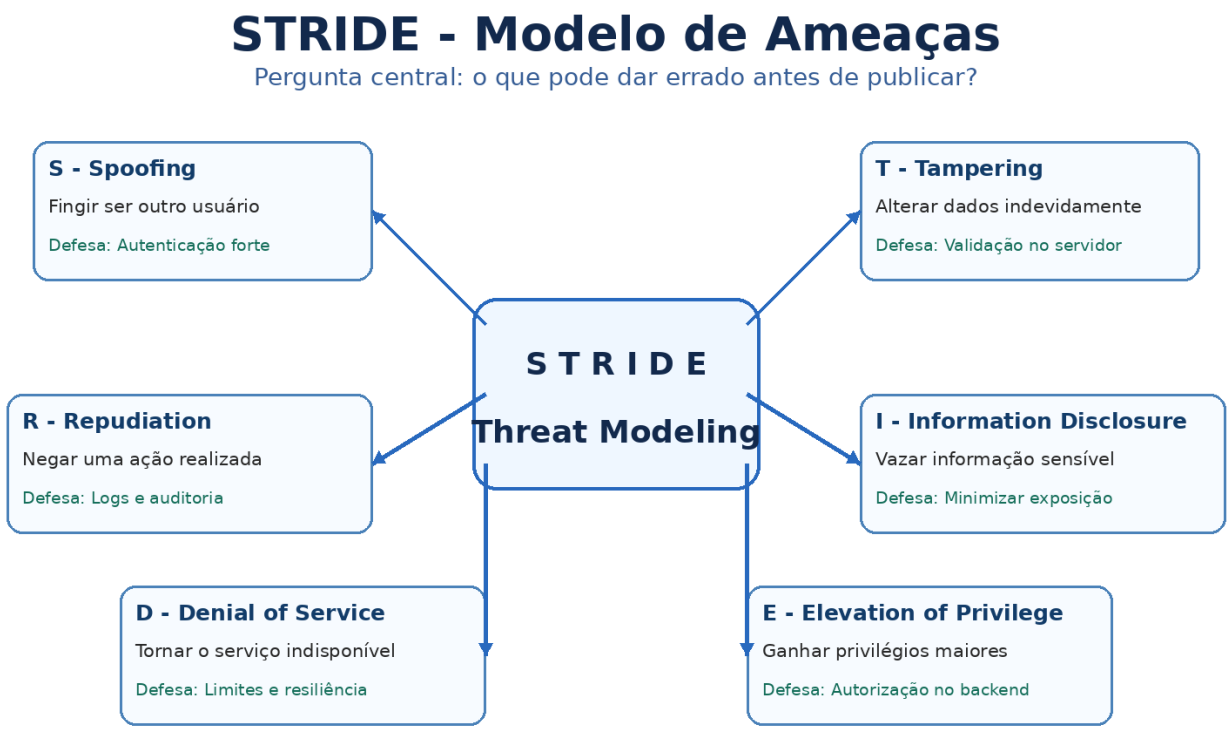
Aula 39 - DevSecOps

Aula 40 - Projeto Final - Pentest de Laboratório

Objetivo deste bloco: sair da visão de "achar bugs" e aprender a prevenir, detectar, validar, corrigir e retestar falhas de segurança.

Aula 36 - Threat Modeling com STRIDE

Threat Modeling é o processo de pensar nas ameaças antes de o sistema estar pronto. Em vez de esperar a falha aparecer em produção, analisamos a arquitetura, os dados, os usuários e as fronteiras de confiança e perguntamos: "o que pode dar errado?".



Mapa visual do STRIDE e exemplos de defesa.

As seis categorias

- S - Spoofing: alguém finge ser outra identidade. Defesa: autenticação forte, sessão segura e validação de tokens.
- T - Tampering: dados são alterados indevidamente. Defesa: validação no servidor e integridade dos dados.
- R - Repudiation: alguém realiza uma ação e depois nega. Defesa: logs de auditoria confiáveis.
- I - Information Disclosure: informação sensível aparece para quem não deveria. Defesa: minimizar dados e erros detalhados.
- D - Denial of Service: o serviço fica indisponível. Defesa: rate limit, timeout, filas e limites de recursos.
- E - Elevation of Privilege: usuário obtém privilégios maiores. Defesa: autorização no backend e menor privilégio.

Exemplo 1 - identidade validada no servidor

JavaScript / Node.js

```
app.get("/profile", authenticate, async (req, res) => {
  const user = await db.users.findById(req.user.id);
  res.json(user);
});
```

Portugol (equivalente didático)

```
algoritmo "perfil_seguro"
var
  usuarioId: inteiro
inicio
  usuarioId <- usuarioAutenticadoId()

  se usuarioId = 0 entao
     escreva("Não autenticado")
  senao
     escreva("Buscar perfil do usuário ", usuarioId)
  fimse
fimalgoritmo
```

O ponto importante não é a sintaxe: a identidade usada pelo servidor deve vir da autenticação validada, não de um userId livre enviado pelo navegador.

Exemplo 2 - não vazar erro interno

JavaScript / Node.js

```
try {
  await processarPedido();
} catch (error) {
  logger.error({ error });
  res.status(500).json({ error: "Erro interno" });
}
```

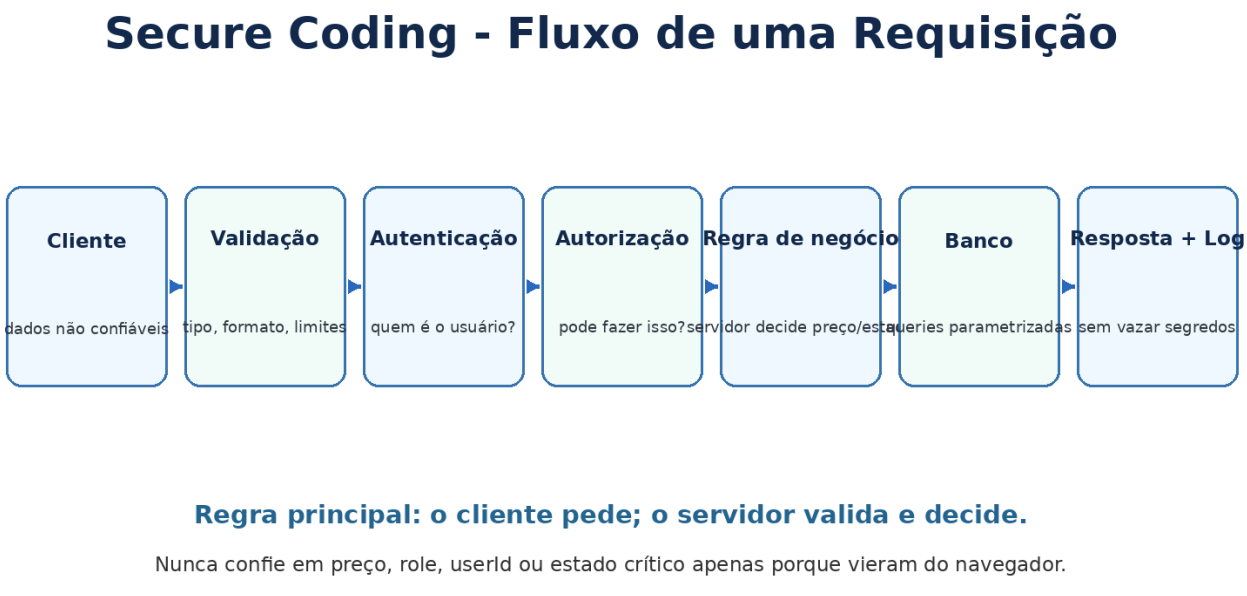
Portugol (equivalente didático)

```
algoritmo "erro_sem_vazamento"
inicio
  se ocorreuErro() entao
     registrarLogInterno("detalhes do erro")
     escreva("Erro interno")
  fimse
fimalgoritmo
```

Pergunta de prova/entrevista: STRIDE não é uma ferramenta de scanner. É um modelo para organizar ameaças durante o desenho e a revisão do sistema.

Aula 37 - Secure Coding

Secure Coding é programar assumindo que toda entrada externa pode estar errada, incompleta ou manipulada. O backend deve validar identidade, permissão, formato, limites e regras de negócio antes de executar ações importantes.



Fluxo de uma requisição segura: o cliente pede; o servidor valida e decide.

Princípios essenciais

- Nunca confiar em preço, role, userId ou estado crítico apenas porque vieram do cliente.
- Validar tipo, tamanho, formato e limites de entrada.
- Usar consultas parametrizadas no banco de dados.
- Armazenar senhas com algoritmo de hash próprio para senhas, como bcrypt/Argon2.
- Guardar segredos em variáveis de ambiente ou serviços de secrets, não no repositório.
- Aplicar autorização em cada operação sensível.

Exemplo 1 - validação com Zod

JavaScript / Node.js

```
const { z } = require("zod");

const ProductSchema = z.object({
  productId: z.number().int().positive(),
  quantity: z.number().int().min(1).max(100)
});

const data = ProductSchema.parse(req.body);
```

Portugol (equivalente didático)

```
algoritmo "validar_produto"
var
```

```
  produtoId, quantidade: inteiro
inicio
  leia(produtoId)
  leia(quantidade)

  se produtoId <= 0 entao
     escreva("Produto inválido")
  senao
     se quantidade < 1 ou quantidade > 100 entao
        escreva("Quantidade inválida")
     senao
        escreva("Dados válidos")
     fimse
  fimse
fimalgoritmo
```

Exemplo 2 - SQL parametrizado

JavaScript / Node.js

```
const result = await db.query(
  "SELECT * FROM users WHERE email = $1",
  [email]
);
```

Portugol (equivalente didático)

```
algoritmo "consulta_parametrizada_conceitual"
var
  email: caractere
inicio
  leia(email)
  escreva("Enviar comando SQL separado do valor do email")
  escreva("O valor não é concatenado dentro do comando")
fimalgoritmo
```

Exemplo 3 - senha e segredo

JavaScript / Node.js

```
const bcrypt = require("bcrypt");

const hash = await bcrypt.hash(password, 12);
const apiKey = process.env.API_KEY;
```

Portugol (equivalente didático)

```
algoritmo "senha_e_segredo_conceitual"
var
  senha: caractere
inicio
  leia(senha)
  escreva("Gerar hash seguro da senha")
  escreva("Ler chave secreta do ambiente, não do código")
fimalgoritmo
```

Secure Coding não é uma única função. É uma sequência de decisões corretas espalhadas por autenticação, autorização, validação, banco, logs, arquivos e configuração.

Aula 38 - SAST, DAST, SCA e Secret Scanning

Ferramentas automáticas enxergam partes diferentes do risco. Um resultado automático deve ser tratado como evidência ou hipótese a ser revisada, e não como verdade absoluta.



Quatro camadas comuns de automação de segurança.

O que cada categoria analisa

- SAST - analisa código-fonte e procura padrões inseguros sem precisar executar a aplicação.
- DAST - analisa a aplicação rodando, enviando requisições e observando respostas.
- SCA - inventaria dependências e compara versões com vulnerabilidades conhecidas.
- Secret Scanning - procura chaves, tokens, senhas e outros segredos expostos em código e histórico.

Exemplo 1 - padrão que SAST pode sinalizar

JavaScript / Node.js

```
app.post("/executar", (req, res) => {
  // Exemplo inseguro: não faça isso.
  const resultado = eval(req.body.code);
  res.send(String(resultado));
});
```

Portugol (equivalente didático)

```
algoritmo "entrada_nao_deve_virar_codigo"
var
  entrada: caractere
inicio
  leia(entrada)
  escreva("Não executar a entrada do usuário como instrução")
  escreva("Tratar entrada como dado, não como código")
```

```
fimalgoritmo
```

Exemplo 2 - dependências

Terminal / npm

```
npm audit
npm outdated
```

Portugol (ideia equivalente)

```
algoritmo "auditar_dependencias_conceitual"
inicio
  escreva("Listar bibliotecas e suas versões")
  escreva("Comparar versões com base de vulnerabilidades")
  escreva("Priorizar atualização quando houver risco")
fimalgoritmo
```

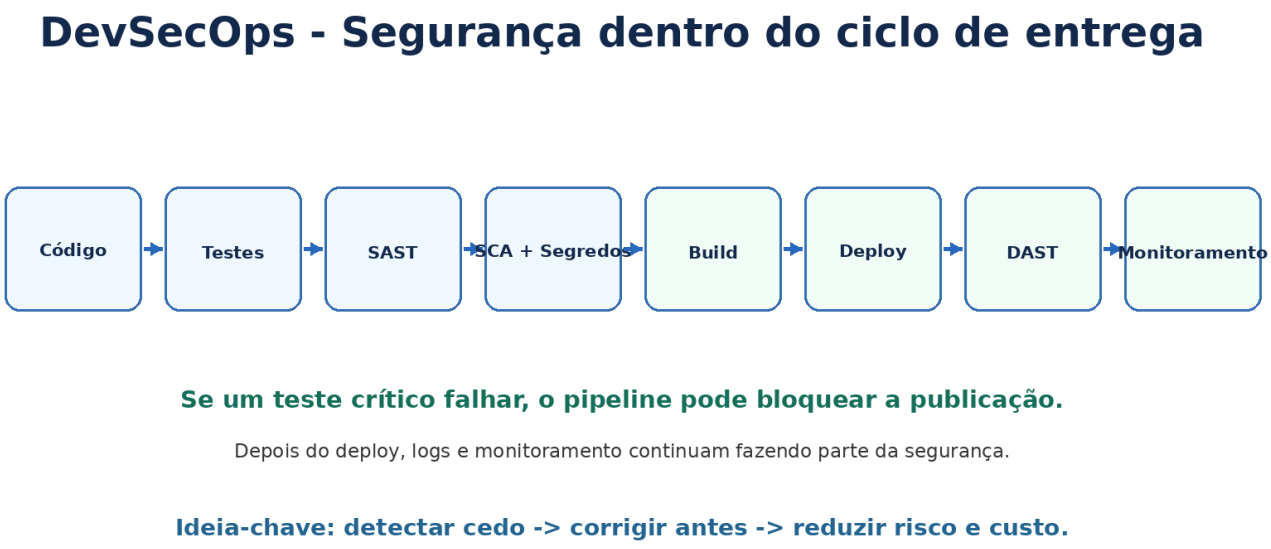
Segredo exposto: o que fazer

- Revogar o segredo comprometido.
- Gerar um novo valor.
- Remover do código e da configuração pública.
- Revisar histórico Git e logs para entender exposição.
- Adicionar secret scanning no pipeline.

Importante: apagar uma chave de um arquivo depois do commit não garante que ela desapareceu do histórico do repositório.

Aula 39 - DevSecOps

DevSecOps integra segurança ao ciclo de desenvolvimento. A meta é detectar problemas o mais cedo possível e continuar monitorando depois do deploy.



Pipeline conceitual de DevSecOps.

Pipeline típico

- Commit/push de código.
- Testes funcionais e unitários.
- SAST, SCA e secret scanning.
- Build do artefato.
- Deploy controlado.
- DAST no ambiente autorizado.
- Logs, alertas e monitoramento contínuos.

Exemplo 1 - log estruturado sem senha/token

JavaScript / Node.js

```
logger.warn({
  event: "LOGIN_FAILED",
  userId: user?.id ?? null,
  ip: req.ip
});
```

Portugol (equivalente didático)

```
algoritmo "log_de_login"
var
  usuarioId: inteiro
  ip: caractere
inicio
  usuarioId <- obterUsuarioId()
  ip <- obterIp()
```

```
  escreva("Registrar: LOGIN_FAILED, usuário=", usuarioId,
          ", ip=", ip)
  escreva("Não registrar senha nem token completo")
fimalgoritmo
```

Exemplo 2 - bloqueio conceitual de pipeline

JavaScript / Node.js

```
if (criticalFindings > 0) {
  console.error("Deploy bloqueado");
  process.exit(1);
}

console.log("Deploy permitido");
```

Portugol (equivalente didático)

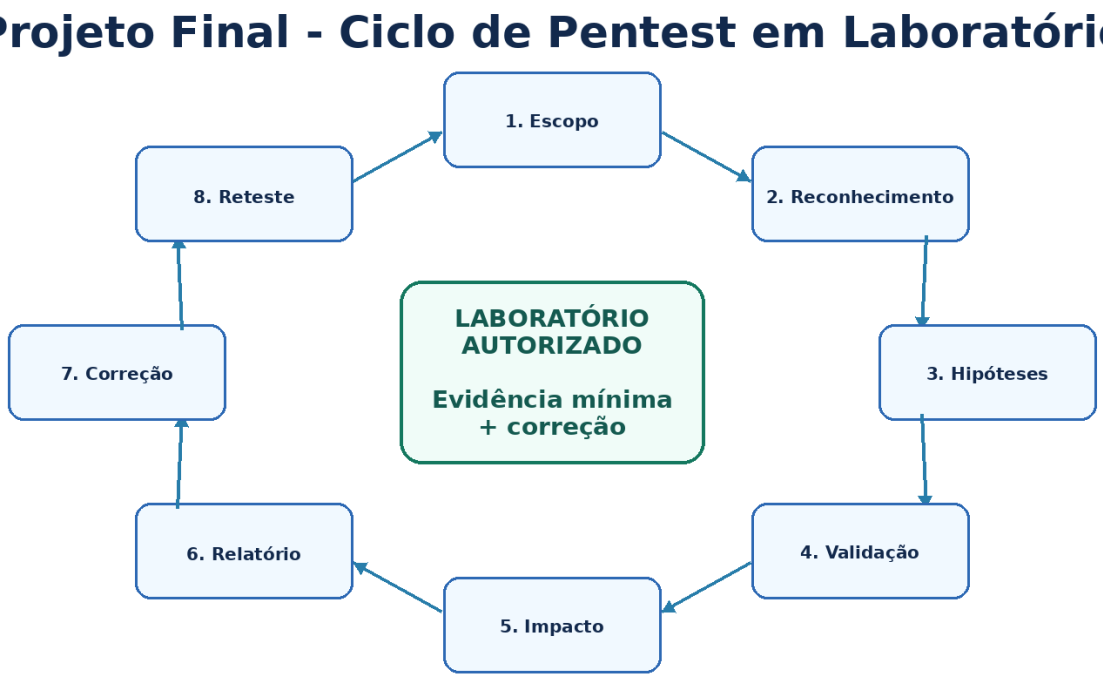
```
algoritmo "gate_de_seguranca"
var
  criticos: inteiro
inicio
  leia(criticos)

  se criticos > 0 entao
     escreva("Deploy bloqueado")
  senao
     escreva("Deploy permitido")
  fimse
fimalgoritmo
```

Shift left = mover verificações para mais cedo no desenvolvimento. Não significa abandonar testes depois do deploy.

Aula 40 - Projeto Final: Pentest Completo de Laboratório

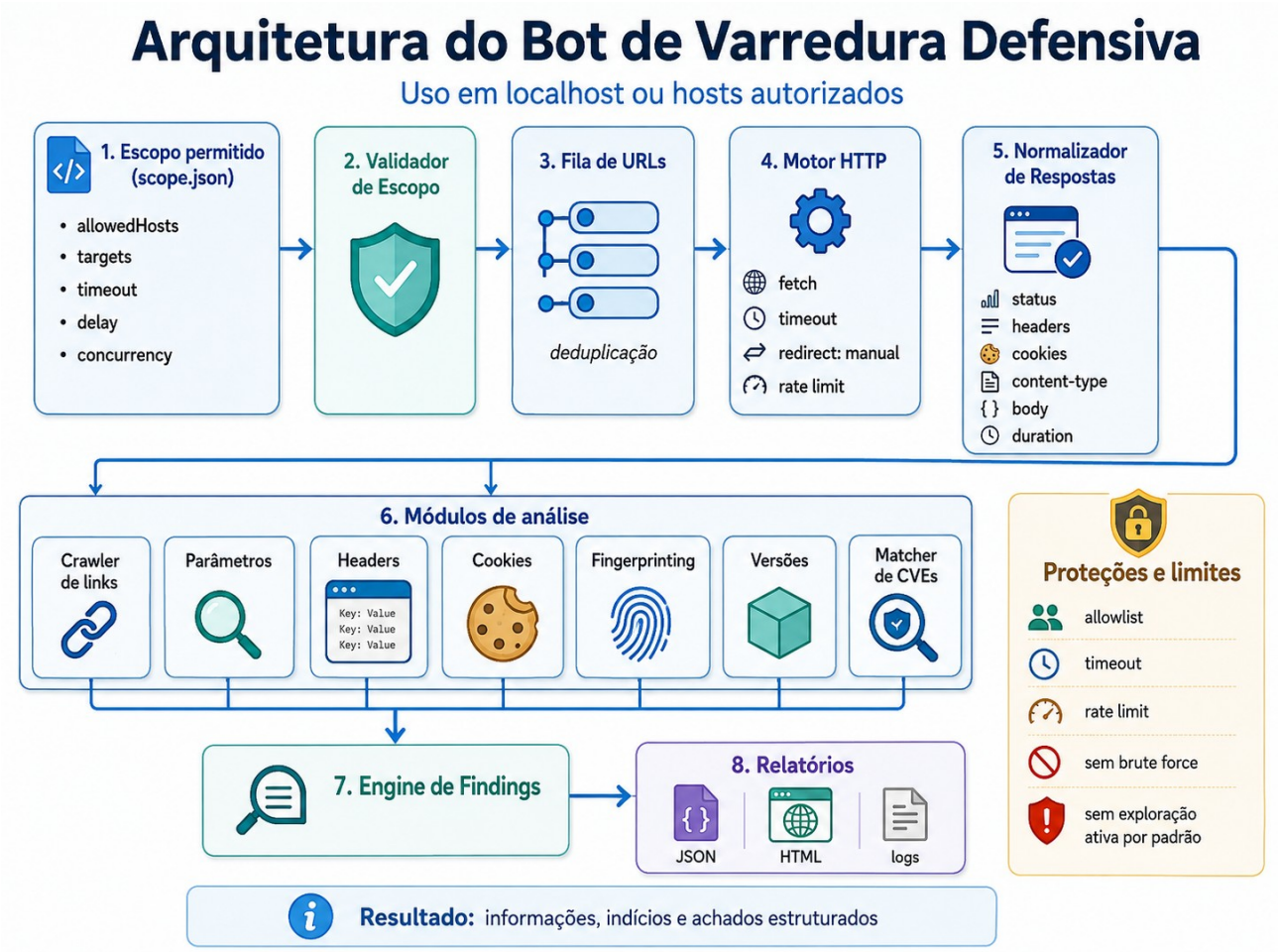
O projeto final junta programação, análise de aplicação, scanner, metodologia de pentest e correção. O alvo deve ser um laboratório local ou um ambiente para o qual exista autorização explícita.



Ciclo recomendado: escopo, análise, correção e reteste.

Arquitetura sugerida

- Aplicação Web em localhost.
- API com autenticação, usuários e pedidos.
- Banco de dados de laboratório.
- Scanner defensivo limitado pelo scope.json.
- Relatório com evidências e severidade.
- Reteste após as correções.



O scanner pode apoiar reconhecimento, inventário e achados defensivos, sempre dentro do escopo.

Exemplo 1 - autorização por proprietário

JavaScript / Node.js

```
app.get("/api/orders/:id", authenticate, async (req, res) => {
  const order = await db.orders.findById(req.params.id);

  if (!order || order.userId !== req.user.id) {
    return res.sendStatus(404);
  }

  res.json(order);
});
```

Portugol (equivalente didático)

```
algoritmo "pedido_com_autorizacao"
var
  pedidoId, donoDoPedido, usuarioAtual: inteiro
inicio
  leia(pedidoId)
  usuarioAtual <- usuarioAutenticadoId()
  donoDoPedido <- buscarDonoDoPedido(pedidoId)

  se donoDoPedido <> usuarioAtual entao
     escreva("Recurso não encontrado ou não autorizado")
  senao
     escreva("Exibir pedido")
```

```
  fimse
fimalgoritmo
```

Exemplo 2 - finding estruturado

JavaScript / Node.js

```
const finding = {
  title: "Controle de acesso inadequado",
  severity: "HIGH",
  endpoint: "/api/orders/:id",
  status: "CONFIRMADO",
  evidence: "Evidência mínima do laboratório",
  recommendation: "Validar proprietário no backend"
};
```

Portugol (equivalente didático)

```
algoritmo "finding_estruturado"
inicio
  escreva("Título: Controle de acesso inadequado")
  escreva("Severidade: ALTA")
  escreva("Endpoint: /api/orders/:id")
  escreva("Status: CONFIRMADO")
  escreva("Recomendação: validar proprietário no backend")
fimalgoritmo
```

Estrutura do relatório final

- Título e severidade do achado.
- Endpoint ou componente afetado.
- Pré-requisitos.
- Passos de reprodução no laboratório.
- Resultado observado e resultado esperado.
- Impacto.
- Evidência mínima necessária.
- Recomendação de correção.
- Status do reteste: resolvido ou ainda presente.

Regra profissional: prove somente o necessário. Um pentest autorizado busca demonstrar e reduzir risco, não causar dano.

Revisão rápida - Aulas 36 a 40

Tema	Ideia principal
Threat Modeling	Antecipar ameaças antes da implementação; STRIDE organiza categorias.
Secure Coding	Validar no servidor, parametrizar SQL, proteger senhas/segredos e aplicar autorização.
SAST	Código-fonte.
DAST	Aplicação rodando.
SCA	Dependências e versões.
Secret Scanning	Segredos expostos.
DevSecOps	Automatizar segurança no pipeline e continuar monitorando depois do deploy.
Pentest final	Escopo -> recon -> hipótese -> validação -> impacto -> relatório -> correção -> reteste.

Perguntas de fixação

- 1. Qual a diferença entre autenticação e autorização dentro de um threat model?
- 2. Por que um backend não deve aceitar o preço enviado pelo frontend como fonte de verdade?
- 3. Qual a diferença prática entre SAST e DAST?
- 4. Se um token foi publicado no Git, por que apenas apagar a linha pode não resolver?
- 5. Em que ponto um pipeline DevSecOps pode bloquear um deploy?
- 6. Por que um finding deve separar INFO, POSSÍVEL e CONFIRMADO?
- 7. Qual é a função do reteste depois da correção?

Gabarito resumido

- 1. Autenticação prova quem é o usuário; autorização decide o que ele pode fazer.
- 2. Porque dados do cliente podem ser modificados; regras críticas devem ser calculadas no servidor.
- 3. SAST analisa código; DAST analisa a aplicação em execução.
- 4. Porque o segredo pode continuar no histórico Git e em cópias/logs; deve ser revogado e substituído.
- 5. Após verificações automáticas, se políticas definidas detectarem achados críticos.
- 6. Para evitar falsos positivos e distinguir informação, hipótese e evidência suficiente.
- 7. Confirmar se a correção realmente eliminou o problema sem introduzir regressão.